

The Codex Maximum Performance Manual

A plain-language guide to your AI-operated study system: how Codex, Claude, Gemini and NotebookLM cooperate, how Google credits and the \$55 reserve are governed, what happens automatically at night, and exactly what you do each day, each week and on mock day.

Built for one student. Studying outranks the system.

Table of Contents

1. What This System Is (and Is Not)	2
2. Why Codex Is the Control Plane	2
3. How the AI Team Cooperates	3
4. Money: Google Credits and the \$55 Reserve	4
5. Your Morning: START in Three Minutes	5
6. During Study: When AI Talks and When It Shuts Up	5
7. After Each Block and End of Day	6
8. Night Jobs: What Happens While You Sleep	6
9. Weekly and Monthly Reviews	7
10. Mock Days	7
11. Question Drills and the Retest Loop	8
12. Research and NotebookLM	9
13. Memory and How the System Remembers	9
14. Self-Improvement, Carefully Governed	10
15. Safety, Privacy, and Data Routing	10
16. Troubleshooting Quick Cards	11
17. Command Reference	12
18. A Day in the Life, Four Ways	12
19. The Forms	13
20. First-Week Pilot and Deployment Checklist	14

1. What This System Is (and Is Not)

You now own a personal operating system for one specific job: maximizing your CUET-UG 2027 score and converting it into a Delhi University seat, while your AI-receptionist business keeps running and your study hours stay protected. The system is not a chatbot you chat with, and it is not a pile of clever prompts. It is a small, disciplined machine with files it trusts, scripts that do arithmetic, a daily rhythm that protects your attention, and a strict rule that studying always outranks system work. Every mechanism inside it exists to produce one of four outcomes: more correct answers, fewer repeated mistakes, better time allocation, or a better strategic decision.

The system is also deliberately skeptical of itself. It refuses to change strategy because of one bad mock, one good day, or one impressive-sounding AI claim. Every fact it relies on carries an evidence tag and a date, and anything that cannot be verified is written down as UNKNOWN with a plan for verifying it. That honesty is a feature you will depend on in April 2027, when the difference between a real trend and a noisy week decides where your final hours go.

What the system is not: it is not a tutor that talks all day, a dashboard you must maintain, or a second job. If it ever takes more than about 25 minutes of administration in a day, that is treated as a defect and the weekly review is required to simplify it. The ideal day looks like this: open Codex, type START, answer six short questions, study, log one line per block, type REVIEW, sleep.

2. Why Codex Is the Control Plane

Version 4 of this system assumed Claude Code was the primary harness. That assumption is dead, and V5 is built around the replacement: OpenAI Codex, driven through your existing ChatGPT Plus plan. Codex earned the control-plane role on evidence, not habit. It reads an AGENTS.md instruction file at every session start, it can run multiple sub-agents in parallel and collect their results, it has command-gating rules, sandboxed approvals, headless execution for nightly jobs, and built-in session resume. Those are exactly the primitives an orchestrator needs, and they are officially documented rather than community folklore.

There is also an economic reason. Your ChatGPT subscription already includes Codex, so orchestration costs no cash. But the plan runs on five-hour and weekly usage windows, which makes Codex attention the scarcest resource in the whole system. V5 therefore enforces one governing idea: bulk work must leave the Codex process. Whenever a task looks like call-an-API-and-return-JSON, Codex runs a script that asks a cheap or free model instead, then reads back a small result. Codex spends its budget deciding and verifying; scripts spend fractions of a cent producing volume.

1 idea

Codex tokens are scarcest; bulk work leaves the process

Practically, this means your interactive sessions stay short and high-value: the morning check-in, explanations that matter, verification of generated questions, weekly analysis. The overnight question factory, the mock scoring, the tagging and classification, all run outside as deterministic scripts calling cheap engines. If Codex is ever unavailable or your window runs out, the scripts and ledgers keep working, which is why the system cannot be paralyzed by a single provider.

3. How the AI Team Cooperates

The system coordinates four families of intelligence, each with a defined job, and it refuses to waste them. GPT-class models inside Codex are the control plane: orchestration, quantitative reasoning, code, and first-pass verification. Claude models are the independent second opinion: they verify high-stakes output precisely because they come from a different company and were trained differently. Gemini handles volume, very long documents, and images of your handwritten work. NotebookLM is the librarian that answers only from your uploaded sources, with citations. DeepSeek and similar cheap models are the factory hands for generating and classifying in bulk. None of these roles overlap by accident; the routing law assigns each task to the cheapest competent worker.

Two laws prevent the classic wastes. The no-redundancy law says you never send the same task to three models for confidence; multi-model verification happens only when uncertainty is material, stakes are high, and disagreement would change an action. The anti-echo-chamber law says Codex output is never judged by Codex alone: any panel reviewing an important result must include at least one non-OpenAI model. When verification is needed at the Claude level, the system prefers to pay with Google credits through Vertex, where Claude models are officially hosted, so your cash reserve survives longer.

Job	Owner	Why
Orchestration, tools, verification (first pass)	Codex (GPT)	Strongest harness; subscription-included
Independent second opinion	Claude via Vertex credits or API	Different family; anti-echo-chamber law
Long documents, images, volume reasoning	Gemini (free tier / credits)	Long context; cheapest at scale
Source-grounded lookup	NotebookLM	Citation-bound librarian; never canonical
Bulk generation and classification	DeepSeek V4 Flash / Gemini Flash	Off-peak prices; scripts validate output

Job	Owner	Why
Arithmetic, ledgers, scoring	Python scripts	Zero hallucination, zero cost

Who does what, in one table

4. Money: Google Credits and the \$55 Reserve

The project has three money layers, and confusing them is how people get surprised. First, subscriptions you already pay: ChatGPT Plus in India at about two thousand rupees a month covers Codex. Second, five hundred ninety-nine dollars of Google Cloud credit running as a relay across two accounts, verified by you on September eighth, 2026: the first account holds two hundred ninety-nine dollars and expires around December seventh, 2026, and the second holds three hundred dollars and activates that same week, so one account hands over to the other with almost no gap. This credit buys Gemini inference and, importantly, Claude-on-Vortex verification. Third, new cash, where your ceiling is \$55 per month. The system treats that ceiling as an emergency reserve with a gate, not as a spending target.

\$0-9 / month

typical new-cash spend while credits last (cap \$55, gated)

While credits are active, the plan spends almost no cash: maybe one to nine dollars a month for DeepSeek overflow and rounding. The credits are spent down deliberately on compounding assets, chiefly the validated question bank and extracted official sources, before they expire. The first account's ninety-day clock ends around December seventh, 2026, so the heaviest asset-building happens in October and November; the second account then carries December through the mock season, and its own expiry date is recorded on the day it activates, because a second ninety-day clock would end around early March 2027, still ahead of the exam. Those dates go into a ledger that forecasts depletion every night. Nothing in this system trusts a remembered balance.

Spending gates are deliberately annoying so they get noticed. Any new recurring cash expense needs a monthly-review vote. Any one-time spend of ten dollars or more needs the buy-nothing check, meaning someone must first prove the free tier, the subscription, and the credits cannot do the job, and then your explicit approval in chat. Crossing the fifty-five dollar line is recorded as a defect with an automatic simplification review the following week. The deep reserve exists so that provider failures or the post-credit era never panic you; it is not pocket money.

5. Your Morning: START in Three Minutes

Open a terminal, move into the project folder, and run Codex. Type one word: START. Behind that single word the system runs a deterministic check script first, which reads your ledgers and reports only warnings: state freshness, ledger integrity, credit expiry proximity, reservoir mode. Then Codex reads your current state file and asks the check-in. It is exactly six short questions, always the same shape: sleep, energy, hours available today, today's constraint, top confusion, and it fills in yesterday's result itself from the ledger so you never have to remember it.

From your six answers plus the ledgers, the system identifies the day's bottleneck and proposes one highest-value action, with a one-line reason grounded in expected score gain. You approve it or say what you would rather do; both choices are logged. Then the conversation ends and you go study. The morning never becomes a planning meeting, because planning was finished the night before; REVIEW seeds tomorrow's first action at 21:30, so START is execution, not deliberation.

Field	Example answer
SLEEP	7
ENERGY	8
AVAILABLE HRS	5.5
TODAY CONSTRAINT	business call at 4 pm
TOP CONFUSION	limits and continuity
YESTERDAY RESULT	(auto-filled from ledgers)

The complete check-in form

6. During Study: When AI Talks and When It Shuts Up

Retrieval practice and productive struggle are the actual engines of learning, so the system's default behavior during a study block is silence. The AI is allowed to speak at four moments only. First, after you have struggled with one blocker for at least ten minutes without progress, you may ask for an explanation, and the first response is a hint, not an answer; the answer itself arrives only if you ask twice. Second, at the end of a block, to capture your one-line log. Third, during scheduled drill windows. Fourth, whenever you explicitly report an error or confusion, which becomes ledger data.

During timed practice, mock sections, recall attempts, and revision sprints, the AI must not speak at all, and during mocks there is a stricter rule still: the sitting is human-only, and any AI assistance voids the diagnostic value of the mock. Explanations that resolve a confusion are never just conversation; each one ends with an error-log entry carrying a taxonomy code, so your confusions accumulate into a map the drill engine uses. The system also refuses to let you study by chatting: pleasant AI conversation feels like learning and is not, and the review counts it as overhead if it shows up.

7. After Each Block and End of Day

After every study block you paste one line: topic, minutes, questions attempted, correct count, error types, one confusion if any, next step. That is the entire reporting burden of the system. Everything else is computed: mastery estimates, spaced-repetition scheduling, decay checks, and the error taxonomy tallies. If you forget to log, tomorrow's START asks once, without drama, and moves on. Guilt is not a mechanism; adherence data is.

At about nine-thirty in the evening you type REVIEW. The system compares planned versus actual minutes, recounts errors by type, flags repeated mistakes, counts AI usage by tier, checks for any model failures or unjustified premium calls, measures how many minutes the OS itself consumed, and writes a five-line daily meta-review plus tomorrow's first action. The review is capped at ten minutes. Its most important product is not the summary; it is that you never wake up to the question of what to study first.

8. Night Jobs: What Happens While You Sleep

A small scheduler on your Mac runs quiet jobs after you stop working, and knowing them removes any urge to run them by hand. At 22:30 the memory layer compacts: old log lines archive, the state file regenerates, integrity checksums update. At 23:00 the off-peak batch window opens: question generation for tomorrow's drills runs on cheap engines, because your evening in India is exactly when DeepSeek's off-peak prices apply and when the free Gemini tier is idle. At 23:20 the budget script audits cash and credit ledgers and forecasts depletion. At 23:40 a model health probe pings the engines the system depends on. On most nights, at 22:40, the self-improvement engine drafts at most one improvement proposal for the weekly review to judge, and it is forbidden from applying anything itself.

These jobs are deliberately boring and deliberately separate from Codex's interactive quota. If any of them fails, the morning bootstrap reports it in one line and the system degrades gracefully; none of them is allowed to write unvalidated content into the question bank or the ledgers. The night layer is also the reason your mornings can be three minutes: the factory runs while the supervisor sleeps.

9. Weekly and Monthly Reviews

Sunday evening, forty minutes, once a week: the weekly review. Scripts pre-compute everything before you sit down, so the review starts with facts rather than feelings: adherence, minutes by subject, accuracy trend, error patterns, retest pass rates, AI usage, burn against forecast. Codex answers twelve standing questions from that pack, and then you make exactly one change for the coming week. The one-change rule is the system's immune system against thrash; everything else you want to fix goes onto a parking list and waits its turn.

The monthly review, first Sunday, up to ninety minutes, is bigger and colder. It assesses fifteen dimensions, reconciles credit and cash against forecast, runs a red-team pass with a multi-model panel, checks the model registry for price or quota drift, and issues verdicts in a fixed vocabulary: keep, change, remove, add, research, defer. Removing is a first-class outcome. Any mechanism that produced no decision and no score-relevant action in a month is a candidate for deletion, because complexity is the quietest way this kind of system dies.

Ritual	When	Duration	Output
Daily REVIEW	21:30	10 min	5-line meta + tomorrow's first action
Weekly review	Sunday 19:00	40 min	12 answers + one change
Monthly review	1st Sunday	90 min	15 verdicts + money reconciliation
Red-team pass	2x per month	45 min	attack notes + patches
Credit reconciliation	monthly	15 min	balances verified against console

Cadence at a glance

10. Mock Days

Mocks are sacred measurements, so the system protects their integrity with structure. The evening before, Codex prints the timing plan and the checklist, then goes silent. The sitting itself is human-only: no AI, no phone, official clock discipline. The moment you finish, you photograph or transcribe your answers, and the deterministic scorer computes your score in about a minute using the official marking scheme, five marks for a correct answer, minus one for a wrong one, zero for a skip. No AI touches that arithmetic, ever; the score is committed exactly as the exam would commit it.

The evening after a mock is for rest, by rule; heavy analysis the same night distorts judgment. Overnight, cheap engines classify your errors, and one strong-model run produces the interpretation: patterns, not blame. Next morning you see the forecast band update, one intervention, and retests of failed items scheduled forty-eight hours out. One bad mock changes nothing by design; two consecutive out-of-band results trigger a panel before any strategy change, because panicking after noise is how preparation strategies die.

+5 / -1 / 0

2026 marking scheme baseline; 2027 values verified at NTA notification

11. Question Drills and the Retest Loop

Drills are how weaknesses become strengths, and the pipeline is built for quality at near-zero cost. It starts from evidence: an error cluster in your ledger, a failed decay probe, or a topic you have never met. Codex writes a short blueprint for the drill, then cheap engines generate the questions in bulk, off-peak. Every single question then passes a structural validator that checks the schema, confirms exactly one correct option, and rejects duplicates against the existing bank. A different model family than the generator then answers the questions blind; agreement is mandatory. Finally a small random sample is audited by a stronger model. Anything that fails is quarantined for a week and then retired; nothing reaches you unvalidated.

The retest loop closes the pipeline. Forty-eight hours after a drill, fresh items on the same subtopic come back, and a mastery gate requires at least eighty-five percent on fifteen or more questions before a topic is considered learned, and again under time pressure, mixed with other topics, before it is considered installed. Gates are computed by scripts from your ledgers, never by a model's opinion. The economics are absurd by design: a thousand validated questions cost roughly the price of a bus ticket, because the expensive intelligence touches only blueprints and audits.

Layer	Checker	Passes what
L1 Structural	Deterministic script, 100% of items	Schema, single correct key, duplicates
L2 Cross-model	Different family, blind	Correct answer and difficulty agreement
L3 Sample audit	Strong model, 5-10% random	Reasoning quality, trap fairness
Quarantine	7-day holding area	Rejects get fixed, retagged, or retired

Quality gates before a question reaches you

12. Research and NotebookLM

Whenever you need a fact that matters, exam rules, college criteria, model limits, the research funnel runs, and its first gate is value: what decision does this change, what does being wrong cost. Searches prefer primary sources, official pages from NTA, Delhi University, and the providers themselves. Captures are saved locally with URLs and dates so evidence does not rot, extraction runs on cheap models into structured JSON, and only claims that survive a cross-check can be canonized into the system with an evidence tag and a refresh clock. A claim without that chain stays labeled UNKNOWN and may inform but never decide.

NotebookLM is the librarian inside this funnel. It holds your official sources, answers with citations, generates study guides and flashcard seeds, and compares what different documents say. It is excellent at grounded retrieval and deliberately kept away from authority: it is never the canonical memory, never a source of exam rules, and its daily caps, which changed in September 2026, are rechecked monthly. There is still no official NotebookLM API, so automation around it stays optional, untrusted, and never connected to your primary Google session. The honest path is manual queries plus exports, and the system treats that as a feature, not a limitation.

13. Memory and How the System Remembers

The system's memory is files it can audit, not vibes it remembers. Append-only ledgers record everything that happens: study blocks, errors with taxonomy codes, mock results, token and credit spending, decisions with their evidence. Derived views, like the state file, are regenerated from those ledgers and are never hand-edited, because the ledgers are the truth and the views are caches. Each file has exactly one writer, which is how the system avoids the corruption that killed earlier personal-AI experiments: models propose, scripts commit.

Codex also has its own native memory feature, and the system gives it a strict role: session scratch. Anything durable must land in the ledgers through a script before the session ends, and a monthly audit compares what Codex remembers against what the files say, with the files always winning. Context is managed with the same discipline: every session loads only the bootstrap and the state file, larger material arrives in task-specific packs, and noisy work is pushed into sub-agents whose summaries are all the main session sees. Memory that cannot be verified is treated as missing, and stale facts are treated as unknown rather than as false or as true.

14. Self-Improvement, Carefully Governed

The system is designed to get better at serving you, and designed so it cannot go mad doing it. Improvements follow one loop: an observation from real ledgers, a pattern that passes an evidence threshold, a written hypothesis, a proposed change, a time-boxed test, an adversarial red-team pass, a git branch, your approval, then measurement and either retention or rollback. Data updates, like ledger entries and health flags, happen automatically. Governance changes, like editing the bootstrap or the routing law, require you every time. The litmus test is blunt: if the change were wrong, would it merely produce a bad week, or would it corrupt the truth? Only the first kind is allowed to move fast.

Thresholds protect you from overreaction, in both directions. Three independent occurrences or seven sustained days of signal are required before any strategy-relevant change; one bad day, one good mock, one model hallucination, one viral study hack, none of these move the system. And the meta-rule keeps the whole engine honest: if self-improvement activity ever exceeds about five percent of system overhead for a month, improving pauses for a month, because a system optimizing itself instead of your score has exactly backwards priorities.

15. Safety, Privacy, and Data Routing

Your data is classified into four levels, and each level has a fixed list of allowed destinations. Public material, like exam questions and official documents, can go anywhere vetted. Low-sensitivity material, like system configs and anonymous logs, follows the same routes. Personal material, anything that identifies you, your category, your school, your business, is restricted to the harness and to providers with strong terms; it never goes to the cheapest third-country engines. High-sensitivity material, keys, billing, credentials, never leaves your machine at all. The scripts that dispatch bulk work strip identity fields before anything is sent.

Secrets live in environment variables, never in files that git tracks, and a validator scans for leaked keys before every governance commit. Backups are weekly and restorable, and the restore drill runs monthly, because a backup that has never been restored is a hypothesis rather than a backup. The Antigravity proxy that circulates in public repos, which bridges Antigravity-hosted models into other tools, remains not installed in this system: it carries reliability complaints, policy risk to your Google account, and credential exposure, and the legitimate paths to the same intelligence, credits through Vertex or the providers directly, are cheaper than that risk.

Class	Examples	Allowed with
Public	Exam content, official docs	All vetted providers
Low	Configs, anonymous logs	All vetted providers

Class	Examples	Allowed with
Personal	Identity, category, business details	Codex; strong-terms providers only, stripped extracts elsewhere
High	API keys, billing, credentials	Your machine only, never transmitted

Data classes and destinations

16. Troubleshooting Quick Cards

Everything breaks eventually, and the system's promise is that its breakage never cancels your studying. The full recovery matrix lives in the implementation files; these cards cover what you will actually hit. The pattern in every card is the same: say the symptom in one line, the system applies a documented fallback, the day continues.

Symptom	Likely cause	What the system does
Codex window exhausted	5-hour or weekly quota	Degrade to profile with lower effort; finish via scripts; defer bulk to night
Codex will not start	Version or auth glitch	Reinstall via official installer; use desktop app for the day
Gemini batch hits 429	Free-tier daily cap	Switch engine to DeepSeek off-peak; retry chunked
Drill questions look wrong	Generator regression	Batch auto-quarantined; validator report attached; generator demoted
Score looks impossible	Key entry error	Scorer stops on malformed rows; re-export the answer sheet
Credits look wrong	Missed ledger entry	Console is the bank; reconcile export into ledger, fix scripts
NotebookLM caps out	Daily limit changed	Use captured sources with Codex summary; recheck limits page
Everything is down	Internet outage	Offline mode: paper mocks, bank drills, deferral queue; study continues

The cards you are most likely to need

17. Command Reference

Thirteen words run the system, and every one of them also accepts plain language in its place. Type them exactly when you are in a hurry; describe the situation when you are not, and Codex maps it to the same workflow. The natural-language path is not a gimmick: saying I keep making mistakes in quadratic equations triggers the same drill cycle as typing DRILL quadratic, and saying mocks feel unstable pulls the forecast band and proposes one intervention.

Command	What it does
START	Morning check-in and today's single highest-value action
STATUS	Full posture: ledgers, quotas, credits, warnings
PLAN	Adjust today's or this week's plan
DRILL topic	Weakness to validated questions to timed set to retest
EXPLAIN concept	Socratic explanation, logged as confusion data
MOCK	Mock-day protocol with integrity protection
RESEARCH question	Evidence-tagged research funnel
REVIEW	End-of-day meta-review and tomorrow's seed
WEEKLY / MONTHLY	Cadence reviews with the short forms
BUDGET	Cash and credit position with depletion forecast
AUDIT / REDTEAM	System self-audit; adversarial pass on a decision
MEMORY	Compact, verify, or archive ledgers

The complete command language

18. A Day in the Life, Four Ways

Four realistic days, with times derived from the actual architecture. On a normal day you wake near seven, type START at 07:01, answer six questions by 07:04, and by 07:07 you are doing the first Maths block on coordinate geometry because the ledgers say that is where expected score gain is highest. Blocks run fifty minutes with ten-minute breaks; AI stays silent except for one hint at minute thirty-four when you ask twice. The evening business hour is fenced, and at 21:30 the REVIEW prints five lines and tomorrow's first action. Overnight the machines make sixty questions from today's errors.

On a strong day the same skeleton stretches to six hours and the system proposes attempting a mastery gate on a stalled topic. On a recovery day, the day after a mock, you do ninety minutes only: read the patterns that arrived overnight, retest failed items, plan, and stop, because recovery is part of the plan, not a failure of it. On a mock Saturday, the sitting runs 09:00 to 12:00 in total silence, the score appears at 12:05 with zero AI involved, the afternoon is genuinely free, and Sunday morning begins with patterns and one intervention rather than raw marks and mood.

19. The Forms

There are exactly three forms in your life, and together they cost less than four minutes a day, a minute on Sunday, and five minutes a month. If any form ever grows, that is a defect to complain about in the weekly review. Everything not on these forms is either computed from ledgers or asked at the moment it is needed and never again.

Field	Fill with
SLEEP / ENERGY	two numbers out of ten
AVAILABLE HRS	realistic hours today
TODAY CONSTRAINT	one line, or none
TOP CONFUSION	one line, or none
YESTERDAY RESULT	auto-filled; verify if wrong
BLOCK LOGS	one line after each block

Daily (six lines)

Weekly	Monthly
Adherence x/7, avg study h/day	Score trend band (auto), biggest win, biggest leak
Best moment, worst moment	Cash spent, credits burned (auto-checked)
One complaint about the OS	Health line, business hours kept
One request	Nothing else: verdicts are computed

Weekly (seven fields) and Monthly (eight fields)

20. First-Week Pilot and Deployment Checklist

Deployment is one careful evening of about two hours, and the pilot week that follows decides adoption, not enthusiasm. The rule during the pilot is the system's own rule: no twenty changes at once. Days one and two run only **START** and **REVIEW** while you measure how many minutes the OS actually costs you. Day three runs the first full drill cycle and you inspect the questions critically. Day five runs a scored mini-mock whose arithmetic you check by hand once, so the scorer earns trust the boring way. Day six deliberately breaks things: kill the internet mid-block and watch the fallbacks behave. Day seven is your first weekly review and a keep, modify, or remove verdict on every mechanism you touched.

Step	Action
1-3	Install Codex CLI, sign in with ChatGPT, record version; create global and project AGENTS.md plus profiles
4-5	Create project structure; add github and fetch MCP servers; set API keys as environment variables only
6-7	Copy starter scripts; run the test suite until green; fill marking scheme and subject configs (2026 baseline, tagged)
8-10	Seed ledgers with baseline scores; verify both Google accounts' credit type, balance, expiry; load NotebookLM notebooks
11-13	Install night-job schedules; initialize git with private remote and first approved tag; run /status and the first START

Deployment checklist (condensed; full version in file 14)

After the pilot, the system earns its place on your desk the same way it keeps it: by protecting your study time, spending almost nothing, remembering honestly, and never pretending to know something it cannot verify. The ideal experience stays exactly what it was designed to be: open Codex, type **START**, answer the check-in, study, log the outcome, type **REVIEW**, sleep. Everything else happens behind the scenes, and everything behind the scenes is written down.